AI Job Matching System

(نظام المطابقة الذكية للوظائف باستخدام الذكاء الاصطناعي)

Under the Supervision of:

Dr. Mona Arafa

Eng. Fatma Ragab

Prepared by

الاسم	السكشن
(Team Leader) محمد تامر محمد على الشيخ	19
محمد خالد عبد الله الطيب محمود محمد البعلى	19
محمد رضا إبراهيم إبراهيم مصطفى	19
محمد رأفت حسن تهامى حسن	19
محمد داود محمد المشوخى	19
محمد حامد عبد السميع السيد محمد	19

Table of Contents

Click Here To Check The Project Repository On GitHub

ANALYSIS PHASE

1. System Requirements

1.1 Methods for Requirements Gathering

Thorough and systematic requirements gathering is the cornerstone of any successful software engineering project. For the AI Job Matching System, a multi-method elicitation strategy was adopted to ensure comprehensive coverage of stakeholder needs, technical constraints, and operational realities of the modern recruitment domain.

1.1.1 Structured Stakeholder Interviews

In-depth semi-structured interviews were conducted with three primary stakeholder groups:

- **HR Managers & Recruiters:** Five sessions were held with HR professionals from technology, finance, and manufacturing sectors to surface pain points in traditional recruitment workflows — including manual CV screening bottlenecks, inconsistent candidate evaluation criteria, and the difficulty of ranking large candidate pools efficiently.
- **Job Seekers:** Focus groups with recent university graduates and mid-career professionals revealed requirements around personalised job recommendations, transparent matching rationale, resume-feedback mechanisms, and mobile-accessible interfaces.
- **System Administrators:** Technical staff responsible for platform operations provided insight into monitoring, audit logging, user management, and data-compliance enforcement requirements.

1.1.2 Quantitative Surveys

Structured online surveys were distributed to a sample of 120 potential end users — 80 job seekers and 40 employers. Key findings informed the following design decisions:

- 87 % of job seekers reported frustration with irrelevant job recommendations from existing platforms, motivating the ML ranking accuracy target of ≥ 85 %.
- 91 % of employers indicated that automated pre-screening would reduce time-to-hire by an estimated 40–60 %.
- 74 % of respondents cited data privacy as a major concern, reinforcing the GDPR/Law 151 compliance mandate.

1.1.3 Domain & Competitor Analysis

A comparative analysis of leading recruitment platforms — including LinkedIn, Indeed, and Bayt.com — was performed to benchmark feature sets, identify usability gaps, and define differentiating capabilities for the AI Job Matching System. This activity produced a feature gap matrix that directly informed the functional requirements catalogue.

1.1.4 Document Analysis

Existing organisational artefacts were examined, including sample CVs in PDF and DOCX formats, employer job description templates, and recruitment workflow process maps. This analysis guided the design of the NLP-based resume parsing pipeline and the structured job-posting data model.

1.1.5 Prototyping & Iterative Validation

Low-fidelity wireframes and data-flow mockups were presented to stakeholder groups in two review cycles. Feedback from these sessions was systematically captured and used to refine requirements before baseline sign-off, reducing the risk of scope changes in later phases.

1.2 Functional Requirements

The following functional requirements define the observable behaviours and features that the AI Job Matching System must provide. Requirements are categorised by user role and system subsystem.

1.2.1 Job Seeker Requirements

- **FR-JS-01 Account Registration & Authentication:** Job seekers shall be able to register using email/password or OAuth 2.0 via Google/LinkedIn. Multi-factor authentication (MFA) shall be supported.
- **FR-JS-02 Profile Management:** Users shall be able to create, edit, and delete a professional profile containing personal information, education history, work experience, skills taxonomy, and career preferences.
- **FR-JS-03 Resume Upload & Parsing:** The system shall accept resumes in PDF and DOCX formats (max 10 MB). The NLP pipeline shall automatically extract name, contact information, skills, experience, and education from uploaded documents and populate the profile fields accordingly.
- **FR-JS-04 Personalised Job Feed:** The system shall present a ranked list of job postings tailored to the user's profile, with a visible match-score percentage for each position.
- **FR-JS-05 Job Search & Filtering:** Users shall be able to search for jobs by keyword, location, industry, experience level, salary range, and employment type, and save searches for future reference.
- **FR-JS-06 Job Application:** Users shall be able to apply to a job posting with one click (using the parsed profile) or by attaching a tailored resume. Application status tracking shall be available.
- **FR-JS-07 Real-Time Notifications:** Users shall receive in-app and email notifications for new high-match job postings, application status changes, and employer messages.
- **FR-JS-08 Privacy Controls:** Users shall be able to control profile visibility (public, employers-only, or hidden), download all personal data, and request account deletion in compliance with GDPR Article 17.
- **FR-JS-09 Dashboard:** A personalised dashboard shall display application history, match-score trends, recommended skill-development actions, and saved jobs.

1.2.2 Employer Requirements

- **FR-EM-01 Company Account & Verification:** Employers shall register a company account; accounts shall be verified via business email domain or manual admin review.
- **FR-EM-02 Job Posting Management:** Employers shall create, edit, pause, and close job postings. Each posting shall include a structured template covering role title, description, required/preferred skills, experience level, location, salary range, and application deadline.
- **FR-EM-03 AI-Ranked Candidate Pipeline:** Upon posting a job, the system shall automatically generate a ranked shortlist of matching candidates from the active user pool, refreshed daily.
- **FR-EM-04 Candidate Search:** Employers shall be able to search the candidate database using advanced filters and view candidate profiles (subject to visibility settings).
- **FR-EM-05 Application Review Workflow:** Employers shall manage applicants through a configurable pipeline with stages (e.g., Applied, Screening, Interview, Offer, Rejected). Status updates shall trigger automatic notifications to candidates.
- **FR-EM-06 Messaging:** A secure in-platform messaging system shall enable direct communication between employers and shortlisted candidates.
- **FR-EM-07 Analytics Dashboard:** Employers shall have access to aggregate analytics on job posting performance — including number of views, applications, match-score distributions, and time-to-fill metrics.
- **FR-EM-08 Third-Party Integration:** Employers shall be able to import job postings from LinkedIn and Indeed via API integration and export candidate data to ATS platforms.

1.2.3 System / Administrator Requirements

- **FR-AD-01 User Management:** Administrators shall be able to view, suspend, reactivate, and permanently delete user accounts (both job seeker and employer), with full audit logging.
- **FR-AD-02 Content Moderation:** Administrators shall review and approve newly registered employer accounts and flagged job postings. An automated rule-based moderation filter shall flag inappropriate content for human review.
- **FR-AD-03 System Monitoring Dashboard:** The admin panel shall display real-time system health metrics: API response times, active user counts, error rates, queue depths, and infrastructure resource utilisation.
- **FR-AD-04 Data Compliance Management:** Administrators shall access a GDPR compliance dashboard showing data processing activities, pending deletion requests, and data breach incident logs.
- **FR-AD-05 ML Model Management:** Administrators shall be able to trigger model retraining jobs, monitor model performance metrics (accuracy, precision, recall, F1), and roll back to a previous model version if performance degrades.
- **FR-AD-06 Reporting:** The system shall generate scheduled and on-demand reports on platform usage, matching accuracy, and user engagement, exportable in PDF and CSV formats.

1.2.4 AI Core Engine Requirements

- **FR-AI-01 Resume Parsing:** The NLP pipeline (spaCy + Hugging Face Transformers) shall extract structured data from unstructured resume text with a field-level extraction accuracy of ≥ 90 %.
- **FR-AI-02 Semantic Skill Matching:** The engine shall use BERT-based semantic similarity to match candidate skills against job description requirements, accounting for synonyms and related competencies (e.g., 'ML' ↔ 'Machine Learning' ↔ 'Supervised Learning').
- **FR-AI-03 Candidate Ranking:** The ML ranking model (combining collaborative filtering and semantic similarity scores) shall rank candidates for a given job posting with an overall accuracy of ≥ 85 % as measured by precision@10 on a curated evaluation dataset.
- **FR-AI-04 Feedback Loop Integration:** Positive employer actions (shortlisting, messaging, hiring) and negative actions (rejection) shall be captured as implicit feedback signals to continuously improve the collaborative filtering model.
- **FR-AI-05 Explainability:** Each match score shall be accompanied by a human-readable explanation listing the top factors contributing to the score (e.g., 'Strong match: Python skills, 3 yrs backend experience, location preference').

1.3 Non-Functional Requirements

1.3.1 Performance & Scalability

- **NFR-PS-01 Concurrent Users:** The system shall support 50,000 concurrent active users without degradation in response time.
- **NFR-PS-02 API Response Time:** 95th-percentile API response time for read operations shall not exceed 300 ms under normal load, and shall not exceed 1,000 ms under peak load (≤ 2× normal traffic).
- **NFR-PS-03 Resume Processing Throughput:** The resume parsing pipeline shall process at least 200 resumes per minute in batch mode and return a parsed result within 5 seconds for real-time single-document submissions.
- **NFR-PS-04 Horizontal Scalability:** The backend services shall be deployed as Docker containers orchestrated by Kubernetes, enabling horizontal auto-scaling based on CPU and request-queue metrics on AWS EC2.
- **NFR-PS-05 Database Performance:** All primary-key lookup queries on PostgreSQL shall complete within 50 ms. Analytical queries on match scores and user engagement data shall complete within 2 seconds.
- **NFR-PS-06 Availability:** The system shall maintain a minimum uptime of 99.5 % (measured monthly), with planned maintenance windows not exceeding 4 hours per month.

1.3.2 Security & Privacy

- **NFR-SEC-01 Data Encryption:** All data at rest shall be encrypted using AES-256. All data in transit shall be protected by TLS 1.3. AWS RDS encryption and S3 server-side encryption (SSE-S3) shall be enabled by default.
- **NFR-SEC-02 Authentication & Authorisation:** Role-based access control (RBAC) shall be enforced across all API endpoints. JWT tokens shall be short-lived (15-minute access tokens, 7-day refresh tokens). OAuth 2.0 with PKCE shall be used for third-party login flows.

- **NFR-SEC-03 GDPR Compliance:** The system shall implement data minimisation, purpose limitation, consent management, right to access, right to erasure, and data portability in accordance with GDPR Regulation (EU) 2016/679.
- **NFR-SEC-04 Egypt Data Protection Law Compliance:** Processing of personal data of Egyptian residents shall comply with Law No. 151 of 2020, including registration with the Egyptian Data Protection Centre and adherence to cross-border data transfer restrictions.
- **NFR-SEC-05 Vulnerability Management:** Automated dependency vulnerability scanning (e.g., Dependabot) and static application security testing (SAST) shall be integrated into the CI/CD pipeline. No critical CVSS ≥ 7.0 vulnerabilities shall remain unpatched in production for more than 72 hours.
- **NFR-SEC-06 Audit Logging:** All administrative actions, authentication events, and personal data access events shall be logged to an immutable audit trail with a minimum retention period of 12 months.

1.3.3 Accuracy & Reliability

- **NFR-AR-01 ML Matching Accuracy:** The candidate ranking model shall achieve a minimum precision@10 of 85 % on the held-out evaluation benchmark, validated prior to each production deployment.
- **NFR-AR-02 Resume Parsing Accuracy:** Named-entity extraction F1 score for skill, experience, and education fields shall exceed 90 % on the internal test corpus.
- **NFR-AR-03 Fault Tolerance:** The system shall implement circuit breakers and retry logic for all third-party API integrations (LinkedIn, Indeed). Failure of a third-party service shall not cause degradation of core matching functionality.
- **NFR-AR-04 Data Integrity:** Referential integrity constraints shall be enforced at the database layer. All write operations shall be transactional, with rollback on failure.

1.3.4 Maintainability

- **NFR-MT-01 Code Quality:** Backend code coverage shall exceed 80 % (unit + integration tests). Linting shall be enforced via pre-commit hooks (flake8 for Python, ESLint for TypeScript).
- **NFR-MT-02 CI/CD Pipeline:** All code merged to the main branch shall pass automated build, lint, test, and security-scan stages before deployment to production via a fully automated pipeline.
- **NFR-MT-03 Documentation:** All API endpoints shall be documented using the OpenAPI 3.0 specification. Inline code documentation shall follow Google-style docstrings (Python) and JSDoc (TypeScript).
- **NFR-MT-04 Model Retraining:** The ML pipeline shall support incremental retraining on new interaction data at a minimum weekly cadence, with automated model evaluation and conditional promotion to production.

2. Stakeholders

A stakeholder is any individual, group, or organisation that has an interest in, or is impacted by, the AI Job Matching System. Stakeholders are classified as primary (direct system users), secondary (organisational and operational stakeholders), and tertiary (regulatory and societal stakeholders).

Stakeholder	Role & Interest	Influence Level
Job Seekers	Primary users who search for employment opportunities. Their primary interests are personalised and accurate job recommendations, ease of profile/resume management, and privacy of personal data.	High
Employers / Recruiters	Companies and HR professionals who post vacancies and source candidates. Interested in efficient candidate discovery, ranking quality, and streamlined hiring workflows.	High
Platform Administrators	Internal technical/operational staff responsible for system health, user management, compliance enforcement, and ML model operations.	High
Dr. Mona Arafa & Eng. Fatma Ragab	Academic supervisors overseeing technical quality, alignment with software engineering best practices, and adherence to graduation project standards.	High
Benha University — FCAI	Institutional stakeholder; the degree-awarding body. Interested in academic rigor, ethical AI use, and faculty reputation.	Medium
Development Team	Engineers responsible for system design, implementation, testing, and deployment. Interested in clear requirements, achievable scope, and technical feasibility.	High
LinkedIn & Indeed (API Providers)	Third-party platforms whose APIs are integrated for job import/export. Compliance with their API terms of service is required.	Medium
AWS (Cloud Provider)	Infrastructure provider (EC2, S3, RDS, SageMaker). Service reliability and data residency policies directly affect system availability and compliance.	Medium
Egyptian Data Protection Centre	Regulatory authority enforcing Law No. 151/2020. Non-compliance may result in penalties and operational suspension.	High
EU Data Protection Authorities	Supervisory bodies enforcing GDPR for any processing of EU residents' personal data.	Medium
Society / Labour Market	Indirect beneficiary. Improved job-to-candidate matching contributes to reduced structural unemployment and more efficient labour market allocation.	Low

3. Use Case Diagram

3.1 Textual Description of Use Cases

The Use Case model for the AI Job Matching System defines the interactions between four primary actors — Job Seeker, Employer, Administrator, and AI Engine — and the system's functional boundaries. The following subsections describe the primary use cases.

Actors

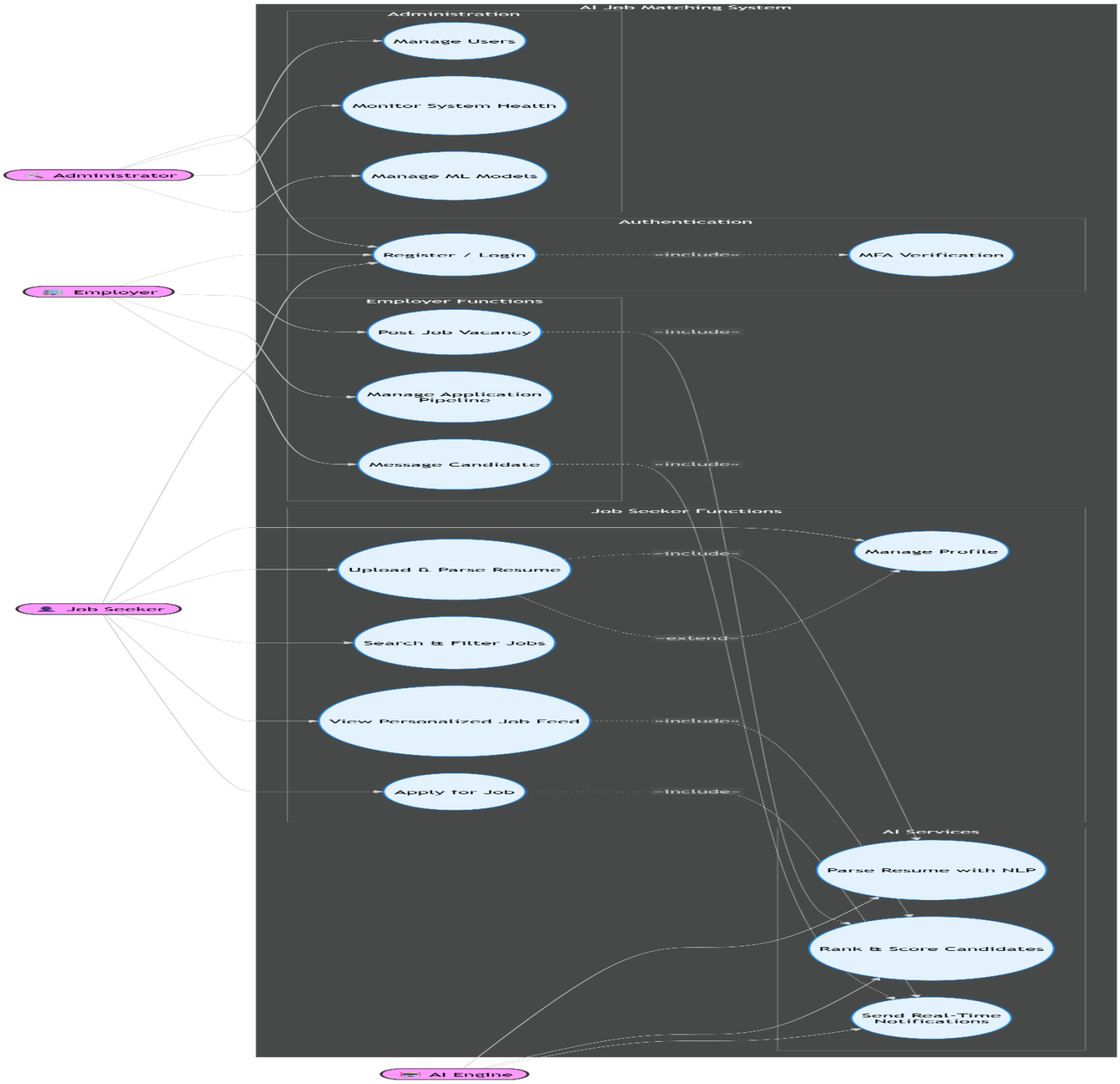
- **Job Seeker:** Any registered individual seeking employment. Interacts with the system to manage their profile, upload resumes, search and apply for jobs, and receive AI-generated recommendations.
- **Employer:** A verified company or recruiter. Interacts with the system to post jobs, review the AI-ranked candidate pipeline, manage applications, and communicate with candidates.
- **Administrator:** Internal privileged user. Responsible for platform governance, user management, compliance oversight, and ML model operations.
- **AI Engine (System Actor):** Represents the automated AI/ML subsystem. Triggered by user-initiated events (resume upload, job posting) or scheduled jobs. Performs NLP parsing, semantic matching, and candidate ranking autonomously.

Primary Use Cases

- **UC-01 Register / Login:** All user types register and authenticate. Includes sub-use cases for OAuth login and MFA verification.
- **UC-02 Manage Profile:** Job Seekers create and maintain their professional profile. Extends UC-03 when a resume is uploaded.
- **UC-03 Upload & Parse Resume:** Job Seeker uploads a PDF/DOCX resume. The AI Engine parses it (includes UC-11) and populates the profile. Extends UC-02.
- **UC-04 Search & Filter Jobs:** Job Seeker searches the job database using keyword, location, and filter criteria.
- **UC-05 View Personalised Job Feed:** Job Seeker views an AI-generated ranked list of recommended jobs. Includes UC-12 (Match Scoring).
- **UC-06 Apply for Job:** Job Seeker submits an application for a specific job. Triggers UC-13 (Notify Employer).
- **UC-07 Post Job:** Employer creates a structured job posting. Triggers UC-12 (Match Scoring) to generate a candidate shortlist.
- **UC-08 Manage Application Pipeline:** Employer reviews, filters, and updates the status of received applications.
- **UC-09 Message Candidate:** Employer sends a message to a shortlisted candidate. Triggers UC-13 (Notify Job Seeker).
- **UC-10 Manage Users:** Administrator views and manages all user accounts, including suspension and deletion.
- **UC-11 Parse Resume (AI):** System actor that performs NLP entity extraction on resume content.
- **UC-12 Rank & Score Candidates (AI):** System actor that generates semantic similarity and collaborative-filtering-based match scores.

- **UC-13 Send Notification:** System actor that dispatches in-app and email notifications upon triggering events.
- **UC-14 Monitor System Health:** Administrator views real-time infrastructure and application metrics.
- **UC-15 Manage ML Models:** Administrator triggers model retraining and monitors accuracy metrics.

3.2 Use Case Diagram Implementation (Click Here to Check the mermaid file)



4. Data Flow Diagram (DFD)

4.1 Context Diagram — Level 0

The Level 0 DFD (Context Diagram) presents the AI Job Matching System as a single high-level process, illustrating the data flows between the system and its four external entities.

External Entity	Data Flows INTO System	Data Flows OUT OF System
Job Seeker	Registration data, login credentials, profile data, resume (PDF/DOCX), job search queries, job applications, feedback actions	Ranked job feed, match scores, application status updates, notifications, parsed profile data, personal data export
Employer	Company registration, login credentials, job posting details, application status updates, messages to candidates, API import requests	Ranked candidate lists, match scores, application data, analytics reports, notifications, export files
Administrator	User management commands, moderation decisions, model retraining commands, configuration changes	System health metrics, compliance reports, audit logs, ML model performance metrics, usage reports
External APIs (LinkedIn/Indeed)	Job posting data (import), candidate profile data (where authorised)	Job posting sync requests, candidate data (export, where authorised by user consent)

4.2 Level 1 DFD — Sub-Process Decomposition

The Level 1 DFD decomposes the single system process into six primary sub-processes, each responsible for a distinct domain of system behaviour. The data stores that persist information between processes are also identified.

Sub-Process P1: User Authentication & Authorisation

Receives login credentials or OAuth tokens from Job Seekers, Employers, and Administrators. Validates credentials against the Users data store, issues JWT access/refresh tokens, and enforces RBAC policies. Outputs an authenticated session context used by all subsequent processes.

Sub-Process P2: Profile & Resume Management

Receives profile data and uploaded resume files from authenticated Job Seekers. Stores raw resume files in the File Storage (AWS S3). Passes resume content to P3 for NLP processing. Reads and writes to the Users/Profiles data store.

Sub-Process P3: AI Resume Parsing (NLP Pipeline)

Triggered by P2 when a new resume is uploaded. Uses spaCy for tokenisation/NER and BERT-based models (via Hugging Face Transformers, deployed on AWS SageMaker) for semantic entity extraction. Outputs a structured JSON object (skills, experience, education) written to the Parsed Profiles data store.

Sub-Process P4: Job Posting Management

Receives structured job posting data from authenticated Employers. Stores job postings in the Job Postings data store. Triggers P5 upon creation or update of a posting. Also accepts job import requests from External APIs and normalises them to the internal schema.

Sub-Process P5: AI Matching & Ranking Engine

The core intelligence sub-process. Reads from the Parsed Profiles and Job Postings data stores. Computes semantic similarity scores using BERT embeddings and collaborative filtering scores from the Interaction History data store. Writes ranked MatchScore records to the Match Scores data store. Triggered by new job postings (P4), profile updates (P2), and scheduled daily refresh jobs.

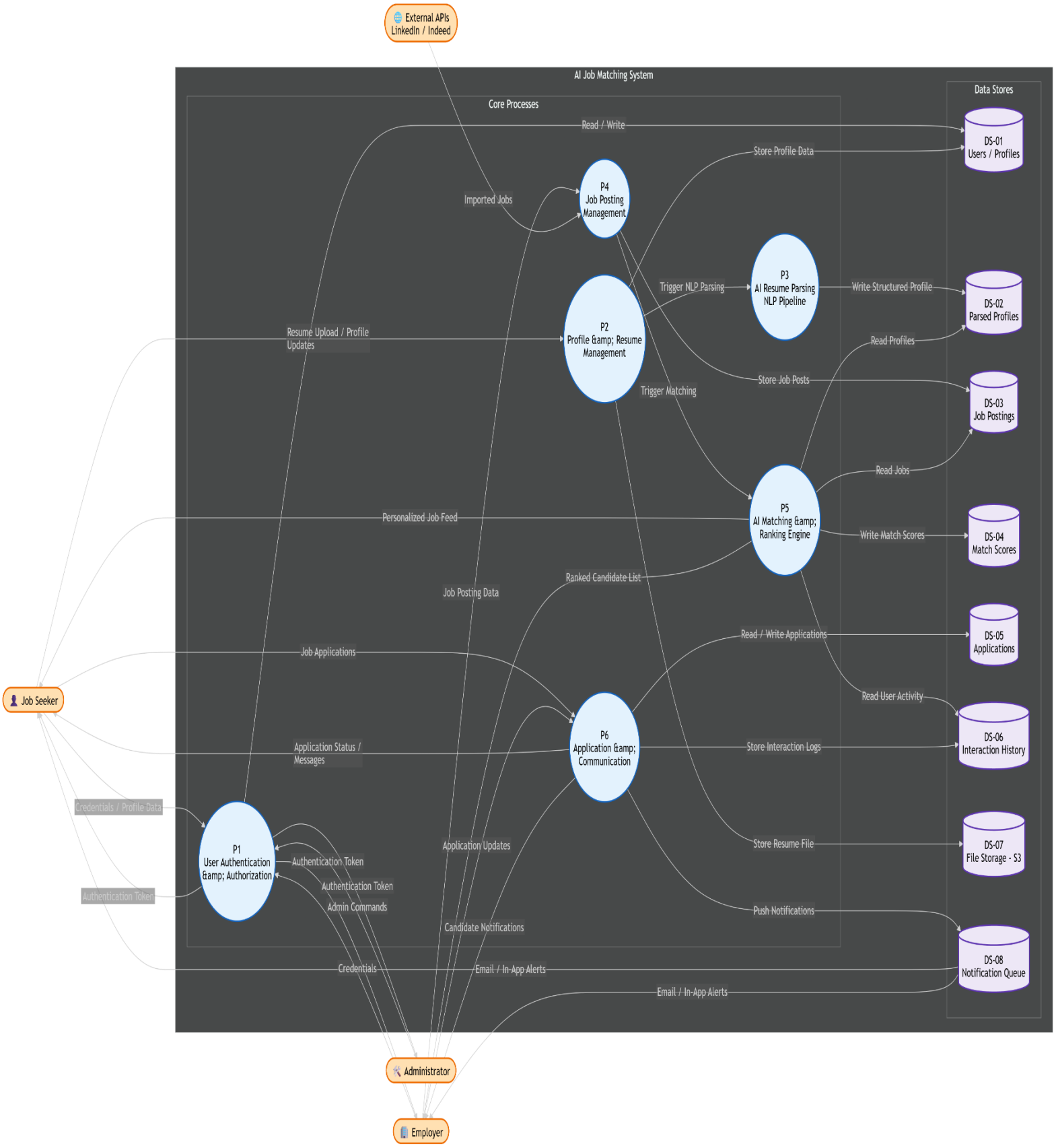
Sub-Process P6: Application & Communication Management

Handles job applications submitted by Job Seekers (reads Job Postings, writes to Applications data store), and manages application status updates by Employers. Invokes the notification service to dispatch real-time alerts to relevant parties. Also handles the in-platform messaging flow between Employers and Job Seekers.

4.3 DFD Data Stores

Data Store ID	Name	Contents
DS-01	Users / Profiles	User accounts, credentials (hashed), profile attributes, privacy settings
DS-02	Parsed Profiles	Structured JSON output of NLP pipeline: skills, experience, education vectors
DS-03	Job Postings	Employer-created job postings including structured requirements and BERT embeddings
DS-04	Match Scores	AI-generated match scores linking candidate profiles to job postings with explainability metadata
DS-05	Applications	Job application records with status, timestamps, and stage history
DS-06	Interaction History	Implicit feedback events (views, applications, shortlists, rejections) used as CF training signal
DS-07	File Storage (S3)	Raw resume files (PDF/DOCX) and uploaded employer assets
DS-08	Notification Queue	Pending notification events for asynchronous delivery via email/in-app

4.4 Level 1 DFD implementation (Click Here to Check the mermaid file)



DESIGN PHASE — DATABASE DESIGN

5. Database Design

5.1 Entity-Relationship Diagram (ERD)

5.1.1 Entities and Attributes

The following primary entities form the conceptual data model of the AI Job Matching System. Each entity's key attributes and its role in the domain are described below.

Entity	Primary Key	Key Attributes	Description
User	user_id (UUID)	email, password_hash, role (ENUM: job_seeker|employer|admin), full_name, phone, is_verified, mfa_enabled, created_at, last_login, is_active	Central identity entity. All system participants inherit from User. Role discriminates behaviour and access rights.
JobSeekerProfile	profile_id (UUID)	user_id (FK), headline, summary, location, years_experience, availability_status, visibility (ENUM: public|employers_only|hidden), linkedin_url, profile_completeness_pct	Extended profile data specific to job seekers. One-to-one with User where role = job_seeker.
Resume	resume_id (UUID)	profile_id (FK), file_s3_key, original_filename, file_format (ENUM: pdf|docx), file_size_bytes, upload_timestamp, parse_status (ENUM: pending|processing|completed|failed), is_active	Stores metadata for uploaded resume files. Raw binary stored in S3; only the S3 key is persisted in the DB.
ParsedProfile	parsed_id (UUID)	resume_id (FK), skills_json (JSONB), experience_json (JSONB), education_json (JSONB), embedding_vector (VECTOR[768]), parse_confidence_score, parsed_at, model_version	NLP-extracted structured representation of a resume. Stores a 768-dimensional BERT embedding vector for

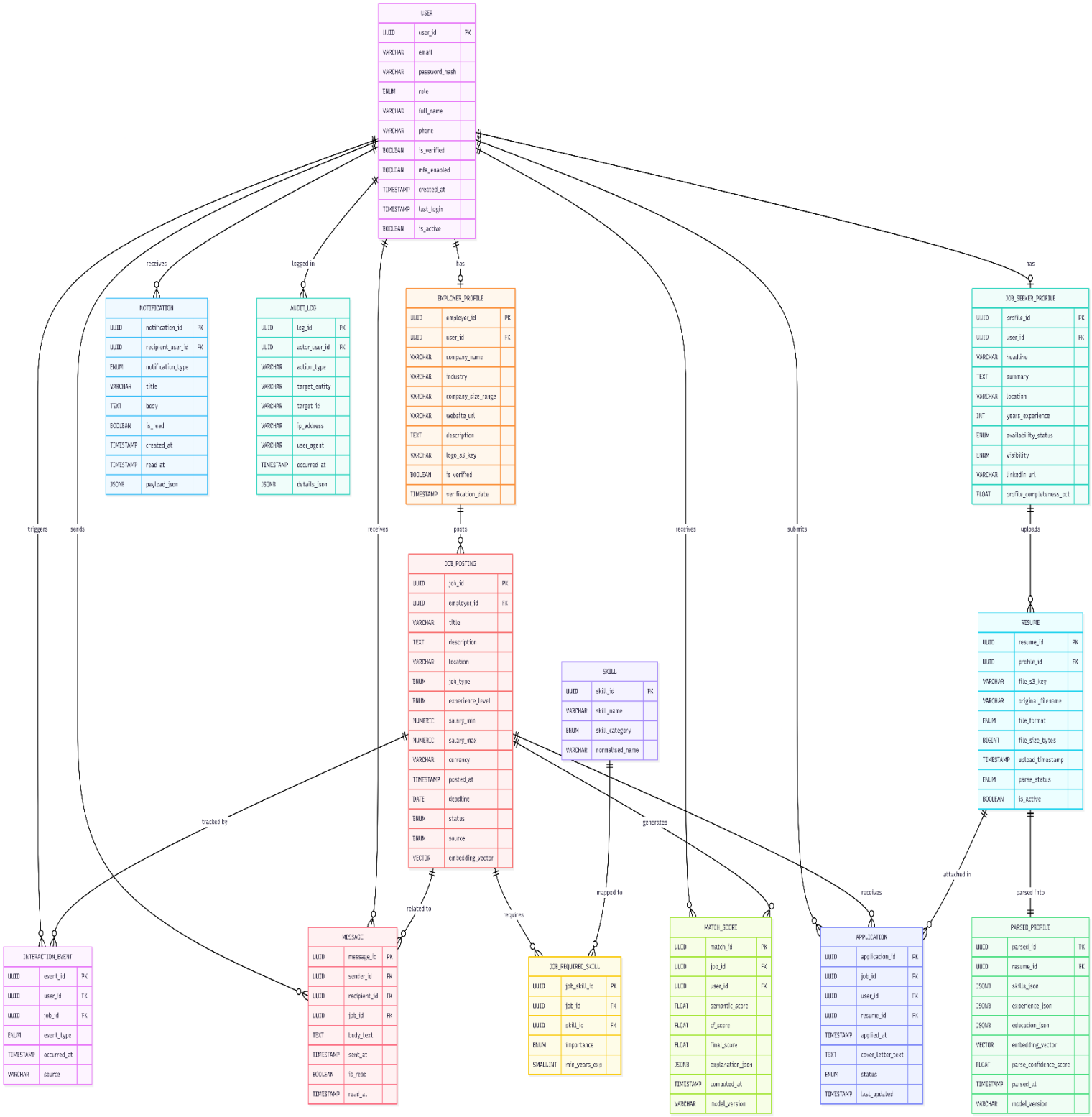
Entity	Primary Key	Key Attributes	Description
			semantic matching.
Skill	skill_id (UUID)	skill_name, skill_category (ENUM: technical|soft|language|domain), normalised_name	Controlled vocabulary of skills used to normalise free-text skill mentions across resumes and job descriptions.
EmployerProfile	employer_id (UUID)	user_id (FK), company_name, industry, company_size_range, website_url, description, logo_s3_key, is_verified, verification_date	Company/employer-specific profile, linked one-to-one with User where role = employer.
JobPosting	job_id (UUID)	employer_id (FK), title, description, location, job_type (ENUM: full_time|part_time|contract|remote), experience_level, salary_min, salary_max, currency, posted_at, deadline, status (ENUM: draft|active|paused|closed), source (ENUM: manual|linkedin|indeed), embedding_vector (VECTOR[768])	Represents an open job vacancy. Stores a pre-computed BERT embedding of the job description for real-time semantic comparison.
JobRequiredSkill	job_skill_id (UUID)	job_id (FK), skill_id (FK), importance (ENUM: required|preferred), min_years_exp	Junction entity resolving the M:N relationship between JobPosting and Skill. Captures the structured requirements of a job.
Application	application_id (UUID)	job_id (FK), user_id (FK), resume_id (FK), applied_at, cover_letter_text, status (ENUM: applied|screening|interview|offer|rejected|withdrawn), last_updated	Records a job seeker's application to a specific job. Drives the employer's application pipeline management.

Entity	Primary Key	Key Attributes	Description
MatchScore	match_id (UUID)	job_id (FK), user_id (FK), semantic_score (FLOAT), cf_score (FLOAT), final_score (FLOAT), explanation_json (JSONB), computed_at, model_version	AI-generated relevance score pairing a job seeker's profile with a job posting. Central artefact of the matching engine.
Notification	notification_id (UUID)	recipient_user_id (FK), notification_type (ENUM: new_match|application_update|message|system), title, body, is_read, created_at, read_at, payload_json (JSONB)	Tracks in-app notifications sent to users, with delivery and read status.
Message	message_id (UUID)	sender_id (FK → User), recipient_id (FK → User), job_id (FK), body_text, sent_at, is_read, read_at	Secure in-platform message between an employer and a job seeker, contextualised to a job posting.
InteractionEvent	event_id (UUID)	user_id (FK), job_id (FK), event_type (ENUM: view|save|apply|shortlist|reject|hire), occurred_at, source	Implicit feedback log used as the training signal for the collaborative filtering model.
AuditLog	log_id (UUID)	actor_user_id (FK), action_type, target_entity, target_id, ip_address, user_agent, occurred_at, details_json (JSONB)	Immutable audit trail for compliance and security monitoring.

5.1.2 Entity Relationships

- **User — JobSeekerProfile (1:1):** One User with role = job_seeker has exactly one JobSeekerProfile. Enforced via unique constraint on profile.user_id.
- **User — EmployerProfile (1:1):** One User with role = employer has exactly one EmployerProfile.
- **JobSeekerProfile — Resume (1:N):** A profile may have multiple resume versions, but only one is flagged as active (is_active = TRUE).
- **Resume — ParsedProfile (1:1):** Each resume has at most one NLP-parsed representation. A 1:1 relationship enforced after parsing is complete.
- **EmployerProfile — JobPosting (1:N):** One employer can post many job vacancies.
- **JobPosting — JobRequiredSkill — Skill (M:N):** A job posting requires many skills; a skill can appear in many job postings. Resolved via the JobRequiredSkill junction entity.
- **User — Application — JobPosting (M:N):** A job seeker may apply to many jobs; each job receives many applications. Resolved via the Application entity.
- **User — MatchScore — JobPosting (M:N):** The matching engine computes a score for each (user, job) pair. Resolved via the MatchScore entity.
- **User — InteractionEvent — JobPosting (M:N):** Many users interact with many jobs. Captured via the InteractionEvent log.
- **User — Notification (1:N):** A user can receive many notifications.
- **User — Message (M:N, self-referential):** Messages have a sender and a recipient, both referencing the User entity.

5.1.3 ERD Implementation (Click Here to Check the mermaid file)



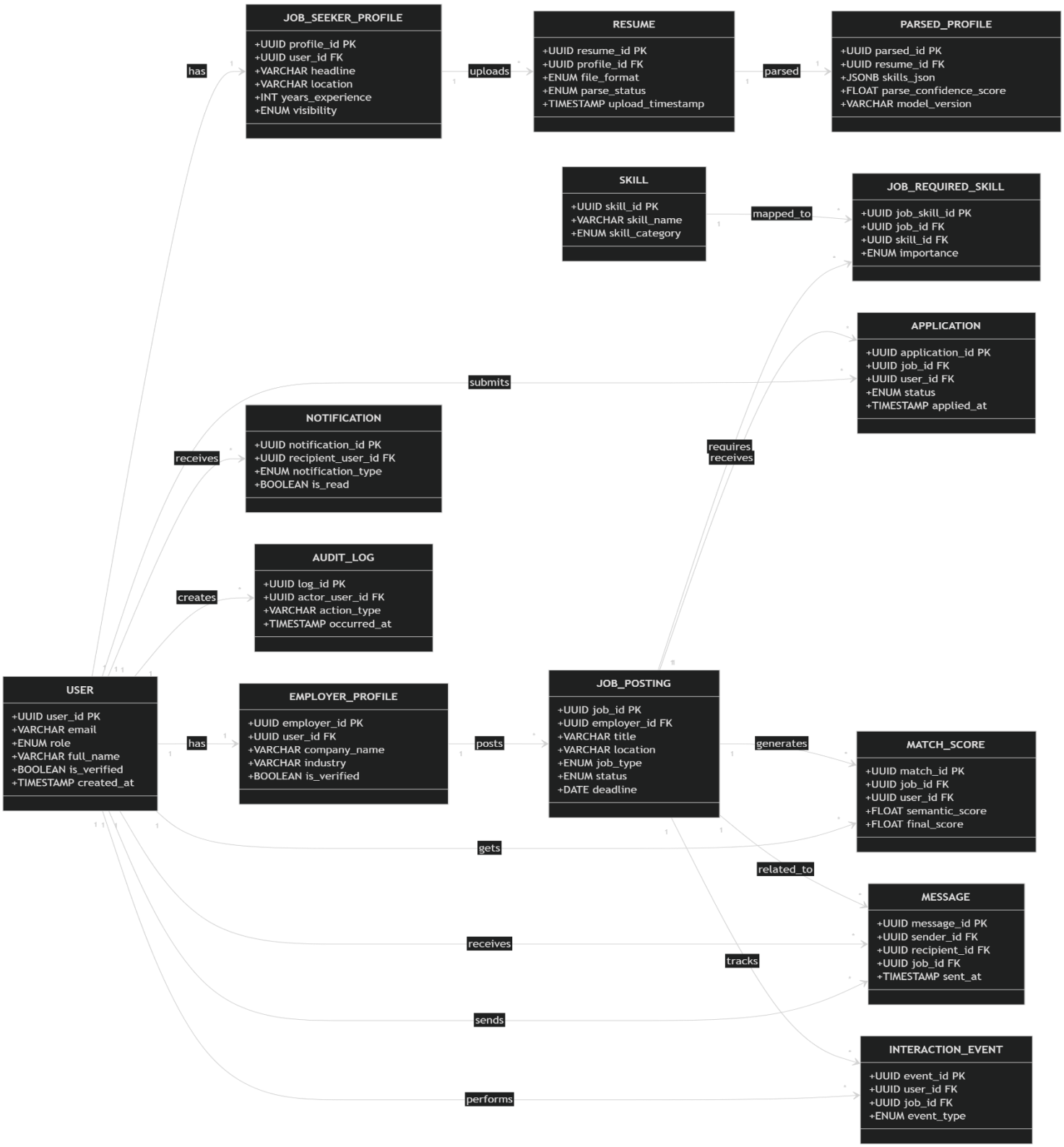
5.2 Relational Mapping Diagram

5.2.1 ERD to Relational Schema Translation

The following section describes how the conceptual entities from the ERD are translated into logical relational database tables. This mapping applies standard normalisation rules (3NF) and adds database-specific artefacts such as indexes and constraints.

- **1:1 Relationships (User → JobSeekerProfile / EmployerProfile):** The foreign key user_id is placed in the sub-entity table (JOB_SEEKER_PROFILE, EMPLOYER_PROFILE) with a UNIQUE constraint to enforce cardinality. A partial index on role ensures that only valid role-type records are associated.
- **1:N Relationships (e.g., JobSeekerProfile → Resume):** The foreign key profile_id is placed in the child table (RESUME). A composite partial index on (profile_id, is_active) supports efficient retrieval of the active resume.
- **M:N Relationships (JobPosting ↔ Skill via JobRequiredSkill):** The junction table JOB_REQUIRED_SKILL carries both foreign keys as a composite primary key (job_id, skill_id), eliminating redundancy.
- **Self-referential Relationship (User → Message):** The MESSAGE table carries two foreign keys referencing USER: sender_id and recipient_id. Both are indexed independently.
- **Vector Columns:** The embedding_vector columns in PARSED_PROFILE and JOB_POSTING use the pgvector extension for PostgreSQL, enabling efficient approximate nearest-neighbour (ANN) queries using IVFFlat or HNSW indexes. Data type is VECTOR(768) for BERT-base embeddings.
- **JSONB Columns:** PostgreSQL's JSONB type is used for semi-structured fields (skills_json, explanation_json, etc.) to balance schema flexibility with queryability via GIN indexes.

5.2.2 Relational Mapping Implementation (Click Here to Check the mermaid file)



End of AI Job Matching System – Analysis Phase